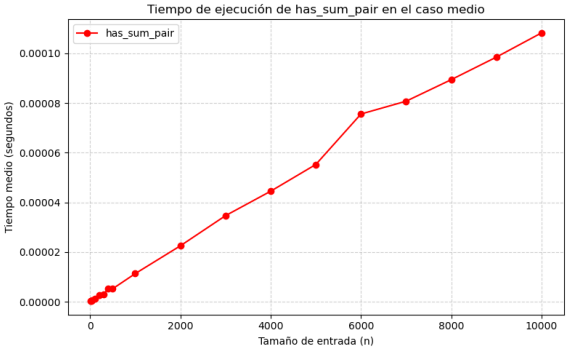
Práctica 1: Python básico. Medición del tiempo de ejecución. TAD Conjunto Disjunto y Componentes Conexas

Algoritmia y Estructuras de Datos Avanzadas
Santiago Pita y Unai Gayoso

1 I.A.4 Medición del rendimiento: `has_sum_pair`

Gráfica de tiempos de ejecución





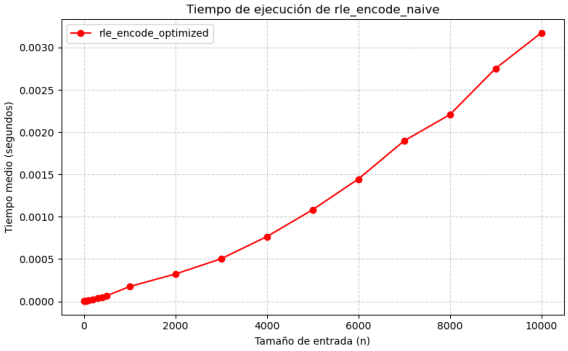
Análisis y Justificación

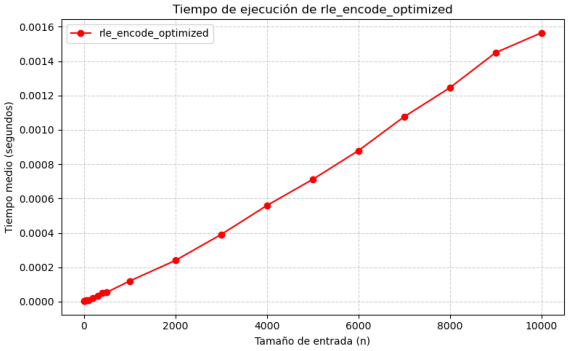
¿Cómo varía el tiempo de ejecución en ambos casos a medida que crece n?
Ambos tiempos de ejecución crecen linealmente, pero el coeficiente que describe la pendiente de la recta del caso medio es menor en algunos puntos casi 4 veces. Esto se debe a que el peor caso siempre recorre entera la lista. Sin embargo, el caso medio la recorre hasta que encuentra el par. Cuantos más pares haya, más probabilidades hay de que salgan al principio y cuanto más al principio menos tiempo tardará.

¿Mantiene la función una complejidad temporal de $\mathcal{O}(n)$ en ambos escenarios?
*La complejidad teórica esperada es $\mathcal{O}(n)$ ya que el algoritmo recorre una lista una sola vez y realiza una búsqueda en un diccionario $\mathcal{O}(n)$ * $\mathcal{O}(1)$. Al observar los resultados empíricos, se obtiene una coincidencia con la teoría. Esto se observa claramente en el crecimiento lineal en las gráficas.*

2 I.B.3 Análisis empírico: Algoritmos de codificación RLE

Gráfica de tiempos de ejecución





Análisis y Justificación

¿Cuál es el comportamiento temporal de ambas funciones al hacer crecer n?
Claramente se observa que la versión naive del algoritmo tiene una complejidad del orden de $\mathcal{O}(n^2)$ *. Esto se debe a que recorre la lista que recibe como argumento una vez y al usar el operador de python '+' tiene que "recopiar" la lista de la salida tantas veces como elementos tenga esta. Sin embargo, la version optimizada que utiliza '.append()' no crea copias sino que concatena los nuevos grupos. Por lo tanto la version optimizada tiene que recorrer solo una vez la lista que recibe como argumento* $\mathcal{O}(n)$*.*

¿Qué impacto tienen las operaciones internas de creación y modificación de listas en Python?
El operador '+' crea una nueva lista y copia todo lo que había en la anterior más los nuevos elementos mientras que '.append()' los concatena en la lista.

3 II.C Cuestiones sobre Conjuntos Disjuntos y Componentes Conexas

1. **Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?**

   No afecta al resultado final. Sí que podemos pasar ramas repetidas o invertidas, pero al procesarlas, el algoritmo comprueba mediante find que ambos vértices ya tienen el mismo representante. Como pertenecen al mismo conjunto, la condición para realizar la unión no se cumple, por lo que el algoritmo no altera la estructura. Por lo tanto, no afecta en absoluto al resultado, ya que el representante es el mismo para todos los nodos que pertenecen al mismo conjunto disjunto.

2. **Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que a su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.**

   Tal y como se explica en la presentación, el algoritmo funciona de forma secuencial: primero inicializa el conjunto disjunto (CD); a continuación, para cada arista (arista) del grafo, busca los representantes del CD de cada elemento del par o vértice. Si no están en el mismo CD (es decir, no comparten representante), los une, ya que sabemos que en el grafo están enlazados. Una vez recorrido toda la lista obtenemos las diferentes componentes conexas como conjuntos disjuntos.

3. **El tamaño de un grafo no dirigido viene determinado por el número n de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.**

   Es del orden de $\mathcal{O}(n)$. Para crear la estructura de destino, el algoritmo recorre una sola vez el rango de los n vértices. Además, recorre una vez la lista de enlaces, que en tiene un tamaño de $n/2$. Por lo tanto, el total es $\mathcal{O}(n) + \mathcal{O}(n/2)$, es de orden de complejidad de $\mathcal{O}(n)$.